

AUDITING STRUCTURED POLICY LEARNING FOR STREAMING TRANSACTION COLLATERAL CONTROL: FACTORIZATION, CONDITIONING, AND CROSS-SCALE TRANSFER

Yingda Yu¹, Sijia Zhou¹, Jiaqi Xuan¹, Zhentong Ye², Guanchao Tong^{1,*}

¹Wenzhou-Kean University

²Northwestern University

*Corresponding author

ABSTRACT

Streaming payment channels must route every arriving transaction into one of k collateral wallets or drop it, periodically flushing a wallet at a fixed fee. We study the resulting collateral-control problem as an online decision problem under twelve traffic regimes. We (i) faithfully port the environment/traffic generator and re-implement three PPO policies—a joint action head, independent factorized heads, and a settle-conditioned factored head. Across five seeds, the settle-conditioned factored head beats the joint head at all four tested capacities, and the independently factored head at three of four (two of four after Holm correction), whereas the extra settle-conditioning path gives *no statistically detectable* gain (paired 95% CIs; zero- and shuffled-conditioning ablations are indistinguishable), a negative mechanism result we report. (ii) We introduce a permutation-equivariant set encoder with no k -dependent parameter, which loads and runs zero-shot across wallet counts where a fixed-shape flat MLP cannot reuse its weights; its transfer quality is asymmetric, with no clear matched-scale advantage at the largest tested k . (iii) A deliberately strong, validation-selected threshold rule attains the highest Money and outperforms the learned policies in nearly all paired comparisons on stationary streams, and incurs zero avoidable drops after abrupt regime shifts. On this balanced workload, wallet fill captures much of the decision-relevant information; we keep this negative rather than hide it. All numbers are produced by released code with fixed seeds and fixed evaluation pools on ported data.

Index Terms— reinforcement learning, structured policies, resource allocation, permutation equivariance, online decision making.

1. INTRODUCTION

Off-chain payment networks such as the Lightning Network [1, 2, 3] process streams of transactions over wallets backed by finite collateral; routing under scarce channel funds has been studied for payment channel networks [4]. Transactions arrive sequentially, each carrying a requested value, and at every arrival the node must decide where to route it: settle it into one of k wallets that currently

holds enough free balance, or reject (drop) it. Independently, it may *flush* one wallet—empty and recharge it at a fixed fee and with a short cooldown during which the wallet cannot accept funds. This is an inherently *sequential* online decision: today’s flush frees capacity for tomorrow’s arrivals but costs money and briefly removes a wallet from service, while greedily accepting now can starve a later, larger transaction. Collateral C is partitioned *per wallet* rather than pooled, so the controller must jointly reason about *where* value lands and *when/what* to empty under non-stationary, bursty traffic with no lookahead to future arrivals. Such sequential decisions are the domain of online competitive analysis [5] and of its learning-augmented variants [6]. The closest formal model, online collateral maintenance with k wallets of capacity C/k , is analyzed with worst-case competitive guarantees in [7]; related online-inventory and learning-augmented packing results include [8].

This structure makes naive policy parameterization costly. A joint categorical controller emits one logit for every (settle, flush) pair, i.e. $(k+1)^2$ outputs at $k = 24$, conflating two decisions that are only weakly coupled and allocating output capacity to invalid or dominated combinations. Branched and factorized action heads have been proposed for exactly this reason [9]. *Factorization* is therefore the natural structural move: predict the settle wallet and the flush wallet as two smaller $k+1$ -way heads, dropping the output count to $2(k+1)$. One can then ask whether the residual coupling matters at all—specifically, whether conditioning the flush decision on the *chosen* settlement wallet (a directed information path settle→flush) adds value, or whether the two choices are already independent under the reward. We treat even that plausible mechanism as a hypothesis to test, not a feature to assume.

Deployment adds a second structural question. The number of wallets k is an operating condition that can change between deployments, yet a flat multilayer perceptron bakes k into its weight shapes: a model trained at one k cannot even be loaded at another. Encoding the wallets as an unordered set with a shared, permutation-equivariant function removes all k -dependent parameters and allows one trained policy to be deployed zero-shot at a new wallet count. Whether such portability comes at an accuracy cost at the original scale is again an empirical question, not a given.

Finally, learned controllers must be audited against a baseline that is actually strong. The prior manuscript’s rule baselines (flush-all, flush-when-full) do not use the most informative signal—current wallet fill. We tune a balance-feedback threshold on validation and put it on equal footing with the learned policies, turning the paper into an audit of *which structural inductive biases genuinely help online collateral control*: does factoring help, does settle-conditioning add anything, does an equivariant encoder buy deployability, and can

AI-assisted development disclosure (AUTHOR CONFIRMATION REQUIRED): an AI coding assistant was used for code development, visualization, and text drafting; quantitative results are generated by released code with fixed seeds and fixed evaluation pools (pytest 37/37 passing; all claims traced to source CSVs); the manuscript prose requires substantive human rewrite and author approval before submission. Final disclosure wording awaits author confirmation. Author contact, ORCID, and funding remain to be filled.

learning beat a cheap feedback rule? We answer these on a faithful port of the prior environment and twelve-regime generator with matched seeds and paired confidence intervals, reporting null results as prominently as positive ones.

Our contributions are:

- A faithful, tested re-implementation of the environment and the twelve-regime traffic generator, and of three PPO policies—**JA-PPO** (joint action over $(k+1)^2$ outcomes), **IFAC** (independent settle and flush heads, $2(k+1)$ outputs), and **SC-FAC** (flush head conditioned on the chosen settlement wallet). The environment is a faithful port; learned policies and unspecified hyperparameters are documented re-implementations.
- A **permutation-equivariant set encoder** (shared per-wallet function + pooling + per-wallet logits, in the spirit of Deep Sets [10]) with no parameter that depends on k , which transfers *zero-shot* across wallet counts; a flat multilayer perceptron cannot (its weight shapes change with k).
- **Mechanism ablations** that zero or shuffle the conditioning signal over matched seeds; they find no detectable Money benefit from the settle→flush path at the tested flush price—an honest null result.
- An **honest strong baseline**: a validation-selected best-fit threshold rule that attains zero avoidable drops, maintains this across the tested regime shifts, and attains the highest Money at every capacity, beating the learned policies in nearly all paired comparisons. We report where learning does—and, more often, does not—beat it.

2. STREAMING COLLATERAL CONTROL

Model. Collateral $C \in \{800, 900, 1000, 1200\}$ is split into $k = 24$ wallets of capacity C/k . At each of $T = 1000$ decisions a transaction of size $x_t \in [0, 1000]$ arrives. The agent picks a settle wallet $a_s \in [0, k]$ and a flush wallet $a_f \in [0, k]$, where index k is a null (no-op). A flush executes *first*, emptying a wallet and making it unavailable for $F=3$ decisions before it refills to full; the transaction is then accepted if a_s is usable and has free balance $\geq x_t$. This order is what couples the two choices in a single step: flushing the wallet into which value is about to settle causes the transaction to be dropped. Drops are attributed as *oversize* ($x_t > C/k$, physically impossible for any policy), *frozen* (cooldown), or *avoidable* (insufficient balance or a flush/settle conflict that a proactive flush could have prevented). Reporting drops by cause lets us separate the structural floor that no controller can remove from the avoidable fraction that measures control quality. The observation is $3k+2$: per-wallet normalized balance, availability, and cooldown, plus x_t normalized and a time-progress scalar (Fig. 1).

Objective. Per-episode Money is $M = p \cdot \sum \text{accepted}_x - \tau \cdot \# \text{flushes}$ with acceptance price $p = 1$ and flush fee $\tau = 10$, so accepting value and spending flushes trade off directly in one scalar rather than through separate metrics. Episodic PPO [11] with generalized advantage estimation [12] optimizes a stochastic policy; evaluation uses the deterministic (argmax) policy on fixed pools.

Data. Episodes are drawn from twelve regimes spanning steady and large-transaction traffic and correlated bursts (calibrated to a target mean), all generated with fixed seeds: 5000 training, 300 validation, and 200 test episodes per regime. Every method and seed is evaluated on the *same* deterministic test pools; the inferential statistical unit is the matched *training seed* ($n=5$ main; $n=3$ abla-

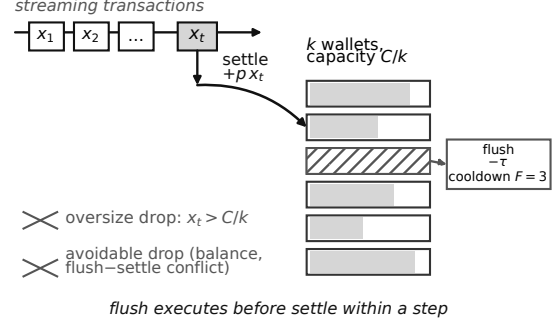


Fig. 1. Streaming collateral control. Each arriving transaction is settled into one of k capacity-partitioned wallets or dropped; the controller may additionally flush one wallet at fee τ . Flush executes before settle.

tion/switching), not the ≈ 2400 test episodes. Hyperparameters absent from the prior manuscript are fixed from validation, never the test set; failed seeds are retained.

3. POLICIES

JA-PPO emits one categorical over all settle/flush pairs: $(k+1)^2$ logits (625 at $k=24$). This treats the step as a single large joint decision, which both inflates the output layer and forces the network to allocate probability mass to combinations that are invalid (flushing and settling the same wallet) or obviously dominated. Structured action spaces have been addressed with large-action embeddings [13], parameterized-action hierarchies [14], action branching [9], and neighborhood construction [15].

IFAC factors the action into two independent $k+1$ categoricals (50 logits), $\pi(a_s, a_f | s) = \pi_s(a_s | s) \pi_f(a_f | s)$. This removes the quadratic joint head, encodes the weak-coupling prior between the two sub-decisions, and shares the trunk representation. It also makes each sub-decision cheaper to learn: most settle/flush combinations are never observed, while each head sees its own signal at every step.

SC-FAC keeps the factorization but routes an embedding of the *chosen* settle wallet into the flush head, $\pi_f(a_f | s, a_s)$, so flushing can in principle depend on where value was just placed. The mechanism it is designed to capture is targeted liquidity management—flushing a wallet other than the one just funded—and is the one “extra” inductive bias whose payoff is genuinely uncertain. Two ablations share its architecture exactly: **no-cond.** zeroes the settle embedding, and **shuffled** feeds a deranged (wrong-wallet) embedding. Comparing the three under matched seeds isolates whether the information path carries any mechanistic value.

Set encoder. The k wallets are intrinsically an unordered set: permuting wallet identities must not change the decision. A flat MLP violates this implicitly by treating wallet slots as an ordered, fixed-length vector, and—more importantly for deployment—ties every hidden weight to the input width and hence to k . We instead encode each wallet with a shared function $h_i = \phi(\text{balance}_i, \text{avail}_i, \text{cooldown}_i, g)$ of its state and two global scalars, pool $\{h_i\}$ symmetrically, and produce per-wallet settle/flush logits (Fig. 2). The map is permutation-equivariant by construction (verified by unit tests), and no layer size depends on k ; this *k-independence* is exactly the property a flat network lacks, and it is what lets weights trained at one wallet count be loaded and executed—without retraining—at an

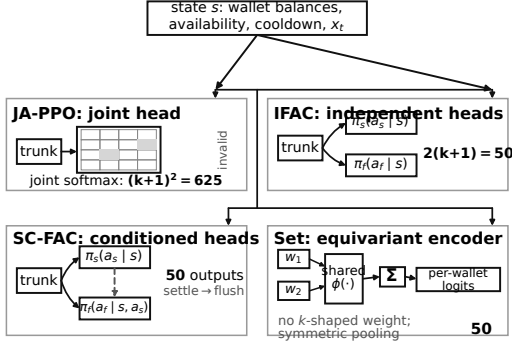


Fig. 2. Action parameterizations. JA-PPO uses a quadratic joint head; IFAC uses two linear factorized heads; SC-FAC adds a settle-conditioned edge to the flush head; the set variant replaces k -shaped trunk weights with a shared equivariant encoder, enabling zero-shot cross- k deployment.

other. The settle-conditioned set variant gathers the chosen settle wallet’s feature into the flush head.

policy	params	logits	latency (ms)
JA-PPO	245,874	625	0.42
IFAC	98,099	50	0.54
SC-FAC	172,883	50	0.62
SC-FAC (no cond.)	172,883	50	0.62
Set-IFAC	135,437	50	0.97
Set-SC-FAC	135,949	50	1.11

Table 1. Parameters, output logits, and CPU latency at $C=1200$, $k=24$. Set-encoder params are independent of k (transferable).

4. EXPERIMENTS

We evaluate deterministic policies on fixed, shared test pools. Model and training choices are made on validation only; failed seeds and null results are retained. *Baselines* are the native one-flush references FA and FWF (reconstructed, since the reference design used multiple flushes per step), a rotation rule, and the strong **BFP0.5**: best-fit routing that proactively flushes the fullest usable wallet once its fill crosses a threshold 0.5, chosen on validation. We also compute an oracle feasible value (transactions that fit at least one wallet) and decompose drops by cause. Results are organized around five questions: whether factorization beats the joint head; whether conditioning adds anything; whether equivariance buys cross- k transfer and at what cost; who is reliable under abrupt regime shifts; and whether the structural changes improve parameter/output efficiency. **Stationary performance: does factorization help?** Table 2 reports Money per capacity and Table 3 the seed-level paired contrasts at $C=1200$; the structural oversize floor is ≈ 400 drops per episode for *every* method, so the differences live in the avoidable fraction. Two effects and one null emerge. *Factorization helps*: SC-FAC significantly outperforms JA-PPO at all four capacities ($p \leq 0.013$, 5/5 seeds; all survive Holm correction), and IFAC at $C \in \{800, 900, 1200\}$ ($p \leq 0.033$) but *not* $C=1000$ ($p=0.57$); after Holm correction IFAC survives only at $C \in \{800, 1200\}$ ($C=900$: $p_{\text{raw}}=0.032$, $p_{\text{Holm}}=0.064$). Mechanism is consistent with: avoidable drops fall from 77/episode (JA-PPO) to 49.5 (IFAC) and 40.4 (SC-FAC), the failure mode the smaller, better-coupled

heads are expected to reduce. *Conditioning does not add Money*: the SC-FAC–IFAC paired difference has a 95% CI including zero at $C \in \{900, 1000, 1200\}$, and at $C=800$ SC-FAC is significantly *lower* than IFAC (-419 , $p=0.009$); SC-FAC accepts marginally more value but spends more flushes, which cancels the gain at $\tau=10$. *A tuned rule beats learning*: BFP0.5 attains the highest Money at every capacity with zero avoidable drops, and its paired advantage is significant in 11/12 method-capacity contrasts; the sole exception is vs. IFAC at $C=1000$ ($+2545$, $p=0.057$, CI crossing zero), which BFP0.5 nevertheless wins on all five seeds. On this balanced i.i.d. workload a single balance-feedback threshold attains the highest observed Money among all evaluated methods, and a 3000-episode PPO budget does not beat it (Fig. 3).

method	$C = 800$	$C = 900$	$C = 1000$	$C = 1200$
FA	2907	4172	5533	8826
FWF	2312	3482	4713	7640
BFP0.5 (rule)	4682	7027	9497	15327
JA-PPO	3080 ± 144	4405 ± 185	6352 ± 36	11893 ± 149
IFAC	4176 ± 69	5465 ± 188	6952 ± 960	13514 ± 96
SC-FAC	3757 ± 43	5404 ± 104	7680 ± 45	13179 ± 259

Table 2. Money per episode (mean $\pm$ SE over $n=5$ training seeds) vs. collateral C . Rules need no seeds; prior-manuscript reported values are not shown (they remain in the reproduction report).

contrast (a–b)	Δ Money	95% CI	p	wins/ n
SC-FAC-IFAC	-336	[-1231, 560]	0.357	1/5
IFAC-JA-PPO	1621	[1108, 2134]	< 0.001	5/5
SC-FAC-JA-PPO	1285	[671, 1900]	0.004	5/5
BFP0.5-SC-FAC	2148	[1428, 2868]	0.001	5/5
BFP0.5-IFAC	1813	[1546, 2079]	< 0.001	5/5
BFP0.5-JA-PPO	3433	[3020, 3846]	< 0.001	5/5
SC-FAC-FA	4353	[3633, 5072]	< 0.001	5/5
IFAC-FA	4688	[4421, 4955]	< 0.001	5/5

Table 3. Seed-level paired Money differences at $C=1200$ (mean $a - b$, paired- t 95% CI, p -value, wins over seeds; statistical unit = matched training seed, $n=5$). Positive favors a .

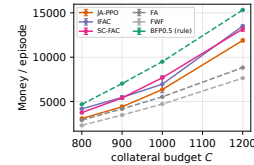


Fig. 3. Money vs. collateral C (solid: learned; dashed: rules). BFP0.5 dominates at every capacity; the factorized policies separate from JA-PPO.

Does settle-conditioning carry mechanistic value? Table 4 compares SC-FAC against the no-conditioning and shuffled-conditioning variants under matched parameters, seeds, and budget. Neither zeroing nor permuting the settle embedding lowers Money detectably: every paired 95% CI includes zero, and at $C=1200$ the fully conditioned variant is nominally the lowest of the three. A correct but useless wiring is not evidence against architecture—it is evidence that, at the tested flush price and observation model, knowing the chosen settle wallet is not decision-relevant once wallet state is already observed.

Does equivariance buy cross- k transfer? We train at $k \in \{6, 12, 24\}$ and deploy, with no retraining, at every k . Fig. 4 shows that the set encoder loads and runs at every deployment k , whereas flat policies have no off-diagonal entry at all because their state-dictionary shapes change with k . The value of the structure is *operational deployability*, not uniformly preserved Money:

variant	Money ($C=1200$, same 3 seeds)	flush cost
SC-FAC	12951±399	571
SC-FAC (no cond.)	13235±100	545
SC-FAC (shuf. cond.)	13252±217	579

Table 4. Settle-conditioning mechanism ablations ($C=1200$; $n=3$ seeds).

zero-shot retention is asymmetric—strong when moving to larger-wallet/smaller- k operating points and weak when a model trained with large wallets meets the small wallets at $k=24$ (e.g. $k=6 \rightarrow 24$ yields 4433). At matched k the set encoder is comparable to or above a flat MLP at $k=6, 12$; at $k=24$ the all-seed aggregate (9190) includes one failed seed (Money=0); excluding it, 13785 vs flat 13610 ($n=3$)—no clear deficit. Structural portability holds; the matched-scale cost is a failed-seed sensitivity, not a stable deficit.

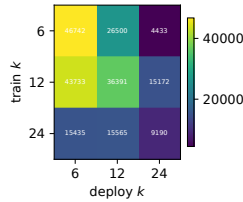


Fig. 4. Cross- k transfer matrix (Set-SC-FAC): rows train k , columns deploy k . Off-diagonal cells are zero-shot (flat MLP has none).

Who is reliable under regime shifts? Trained only on stationary pools, every policy is tested on the *same* deterministic episodes spliced across regimes at an abrupt change point (six scenarios [16]: calm→burst, burst→calm, early and late). Table 5 reports post-switch avoidable drops and Money. BFP0.5 incurs *zero* avoidable drops after every switch and the highest Money, maintaining its observed performance without retraining because current wallet fill captures much of the decision-relevant information for this feedback law. The learned policies generalize far better than the naive rules (28–56 vs. 144–148 post-switch drops). Against BFP0.5, JA-PPO and IFAC remain worse on both drops and Money over the paired scenarios (one-sample paired tests, $n=3$; $p < 0.01$); for SC-FAC the drops contrast is weak ($p \approx 0.09$), and its Money contrast ($p < 0.05$ uncorrected) does not survive Holm correction across the six switching tests ($p \approx 0.07$). IFAC and SC-FAC incur similar post-switch drops (28.3). The defensible reading is that learned policies do not collapse when the stream changes—but we do *not* observe any switch-adaptation advantage of learning over the tuned feedback rule, and we deliberately do not claim one.

policy	Money	post Money	post rec. drops
FA	8585	4232	143.8
FWF	7593	4075	148.4
BFP0.5 (rule)	15319	8206	0.0
JA-PPO	11694	6114	56.0
IFAC	13681	7220	28.3
SC-FAC	13108	7006	28.3

Table 5. Switching streams (mean over six scenarios; learned over three seeds). Post-switch avoidable drops and Money.

Parameter and output efficiency. Table 1 shows the structural pay-off: replacing the joint head cuts outputs from 625 to 50 and the actor from 245,874 to 98,099 parameters (IFAC); the set variants use 135k parameters with *no* k -dependent tensor and sub-millisecond CPU latency. Factorization yields a representation that is smaller

and cheaper while statistically stronger than the joint head on most capacities.

5. DISCUSSION AND LIMITATIONS

The central result is a boundary map for structured policy learning on this task, not a blanket win for RL. *Factorization is the reliable learned effect*: replacing a quadratic joint head with two linear sub-heads is consistent with removing the avoidable drops the joint head was producing, with gains surviving multi-seed paired tests (Holm: all four for SC-FAC, two of four for IFAC) and a simultaneous reduction in parameters and outputs. *Settle-conditioning is a null*: once wallet fill is in the state, the identity of the just-chosen settle wallet carries no additional decision-relevant signal at $\tau=10$, and both the zero and shuffled controls confirm this. We report it as a wiring that is plausible but unremunerative here, rather than quietly dropping it.

Equivariance buys a different currency. The set encoder’s value is operational deployability—shipping one trained policy to a wallet count the training never saw, which the fixed-shape flat architecture cannot do without resizing. Transfer is asymmetric; at $k=24$ the all-seed aggregate is sensitive to one failed seed—excluding it, no clear matched-scale deficit remains. A practitioner at one fixed k should prefer the flat factorized model; one who must redeploy across changing k gains real optionality.

The strong rule is the paper’s uncomfortable finding. On balanced, i.i.d. streams with full state observation, a one-line fill-feedback threshold reaches zero avoidable drops stationary and after every abrupt switch, and beats the learned controllers in nearly every paired comparison. This must not be hidden: wallet fill captures much of the decision-relevant information here, and the residual problem is nearly solved by feedback control. Future learning could earn its place on predictable non-i.i.d. burst structure, heterogeneous wallet sizes or fees, multi-step lookahead under expensive flushes, or reward functions misaligned with accepted value. We do not instantiate an untested heterogeneous environment to manufacture a regime where RL wins.

Limitations: one flush per step, finite horizon, single fixed fee, no lookahead, homogeneous wallets, single-process PPO; CPU training was faster than the available GPU for batch-one rollouts. The prior manuscript’s learned numbers are treated as reported, not regenerated: the environment is a faithful port while policies and unspecified hyperparameters are documented re-implementations chosen on validation.

6. CONCLUSION

We audited which structural inductive biases help a learned streaming collateral controller. Factoring the joint action into separate settle/flush heads gives a reproducible gain—significant at all four capacities for SC-FAC (surviving Holm) and three of four for IFAC (two of four after Holm, $C=900$ non-surviving)—while cutting outputs from $(k+1)^2$ to $2(k+1)$. The settle-conditioning path yields no detectable Money gain under matched-seed zero/shuffle ablations. A k -independent equivariant encoder enables zero-shot cross- k deployment where the fixed-shape flat architecture cannot without resizing, at an asymmetric transfer cost sensitive to one failed seed. Most importantly, a validation-selected feedback rule attains the highest Money with zero post-switch avoidable drops and beats the learned controllers in nearly every paired comparison—we present it as the benchmark future learning must beat.

7. REFERENCES

- [1] Joseph Poon and Thaddeus Dryja, “The bitcoin lightning network: Scalable off-chain instant payments,” Tech. Rep., Technical Report, 2016.
- [2] Andrew Miller, Iddo Bentov, Surya Bakshi, Ranjit Kumaresan, and Patrick McCorry, “Sprites and state channels: Payment networks that go faster than lightning,” in *Financial Cryptography and Data Security (FC 2019)*. 2019, vol. 11598 of *Lecture Notes in Computer Science*, pp. 508–526, Springer.
- [3] Kartick Kolachala, Mohammed Ababneh, and Roopa Vishwanathan, “Sok: Payment channel networks,” *arXiv preprint arXiv:2407.20968*, 2024.
- [4] Vibhaalakshmi Sivaraman, Shaileshh Bojja Venkatakrishnan, Kathleen Ruan, Parimarjan Negi, Lei Yang, Radhika Mittal, Giulia Fanti, and Mohammad Alizadeh, “High throughput cryptocurrency routing in payment channel networks,” in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2020)*. 2020, pp. 777–796, USENIX Association.
- [5] Allan Borodin and Ran El-Yaniv, *Online Computation and Competitive Analysis*, Cambridge University Press, 1998.
- [6] Thodoris Lykouris and Sergei Vassilvitskii, “Competitive caching with machine learned advice,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 2018, vol. 80 of *Proceedings of Machine Learning Research*, pp. 3296–3305, PMLR.
- [7] Ghada Almashaqbeh, Sixia Chen, and Alexander Russell, “Competitive policies for online collateral maintenance,” in *6th Conference on Advances in Financial Technologies (AFT 2024)*, 2024, vol. 316 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pp. 26:1–26:16.
- [8] Qi Lin, Hongjian Yi, Jiashu Pang, Minghua Chen, Adam Wierman, Michael Honig, and Yuanzhang Xiao, “Competitive online optimization under inventory constraints,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems (POMACS)*, vol. 3, no. 1, pp. 1–28, 2019.
- [9] Arash Tavakoli, Fabio Pardo, and Petar Kormushev, “Action branching architectures for deep reinforcement learning,” in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. 2018, vol. 32, pp. 4131–4138, AAAI Press.
- [10] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander J. Smola, “Deep sets,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [12] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel, “High-dimensional continuous control using generalized advantage estimation,” *arXiv preprint arXiv:1506.02438*, 2015.
- [13] Gabriel Dulac-Arnold, Richard Evans, Hado van Hasselt, Peter Sunehag, Timothy Lillicrap, Jonathan Hunt, Timothy Mann, Theophane Weber, Thomas Degris, and Ben Coppin, “Deep reinforcement learning in large discrete action spaces,” *arXiv preprint arXiv:1512.07679*, 2015.
- [14] Matthew Hausknecht and Peter Stone, “Deep reinforcement learning in parameterized action space,” in *International Conference on Learning Representations (ICLR)*, 2016.
- [15] Fabian Akkerman, Julius Luy, Wouter van Heeswijk, and Maximilian Schiffer, “Dynamic neighborhood construction for structured large discrete action spaces,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [16] Joao Gama, Indre Zliobaite, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.